Eventual Consistency in Practice

stale for a moment, correct in the end

Watch the 4-min video

In one line: eventually consistent replicas may briefly disagree but converge once updates stop; it buys latency and availability, and is safe to use when you add session guarantees like read-your-writes, design conflict-free updates, and keep strong consistency for the few things that truly need it.

1 99 or 100?

You like a post and the count goes from 99 to 100. Your friend, on another server, still sees 99 for a few seconds. Nothing is broken: the replicas simply haven't caught up yet. That's **eventual consistency**: if no new updates arrive, all replicas eventually return the same value. Most of the internet runs this way, from DNS to social feeds to shopping carts.

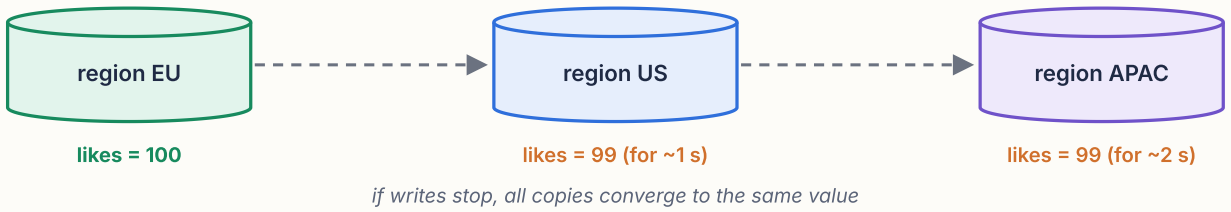


Figure 1. Replicas accept writes locally and copy them asynchronously; for a short window they disagree.

2 Why accept it

Strong consistency needs coordination: a write waits for other replicas, often in other regions, and during a partition some requests must fail (lesson 3). Eventual consistency lets each replica answer immediately, so reads and writes stay fast and available. The price is **staleness** and possible **conflicts**.

3 Making it feel consistent: session guarantees

Users mostly notice inconsistency with their own actions. Four cheap guarantees remove most of the confusion:

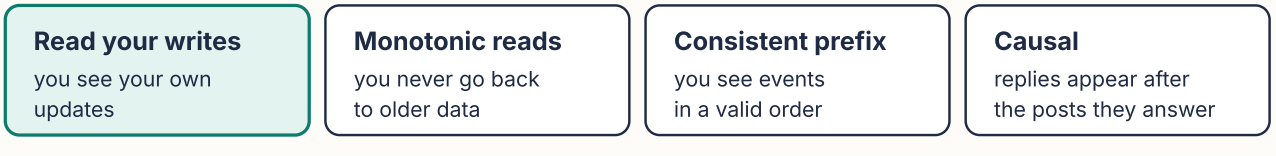


Figure 2. Session guarantees. Read-your-writes alone fixes the most common complaint.

Implement them by routing a user to the same replica (sticky sessions), reading from the leader for a few seconds after that user writes, or passing a version token (the log position of the user's last write) and waiting until the replica has reached it.

WORKED EXAMPLE: LIKE COUNTS

Each like is written to the nearest region's replica, and the count is a **CRDT counter** (lesson 19), so concurrent likes from different regions merge by adding, never by overwriting. The person who liked sees 100 immediately, because the app updates the count locally and reads from the region it wrote to. Others see it within a second or two. The 'did I like this?' flag uses read-your-writes, so the heart never flips back. Payments for promoted posts, by contrast, go through a strongly consistent ledger.

4 Handling conflicts

- **Last-write-wins:** simple, but loses concurrent updates and depends on clocks (lesson 5).
- **Merge functions:** union of cart items, max of timestamps, sum of counters; CRDTs make this automatic.
- **Ask the application or user** to resolve, as with Git merges.
- **Avoid them:** route all writes for one key to one leader.



Figure 3. Consistency is a spectrum. Pick per feature, not per company.

COMMON MISTAKES

- Reading from a replica right after a write and showing the user old data.
- Eventual consistency for balances, inventory limits or unique usernames.
- No monitoring of replication lag, so 'eventually' becomes minutes.
- Last-write-wins on data where merges are easy and losses hurt.

5 Key terms

Eventual consistency: replicas converge if updates stop

Read-your-writes: a user always sees their own updates

Replication lag: how far a replica is behind the leader

Staleness: how old a read may be

Monotonic reads: no going back in time across reads

Convergence: all replicas reach the same state

6 Interview questions

Q1. What is eventual consistency? BASIC

A guarantee that if no new updates are made, all replicas will eventually return the same value; reads in between may be stale.

Q2. What's read-your-writes consistency, and how do you provide it? BASIC

A user always sees their own updates. Read from the leader briefly after a write, use sticky routing, or wait for the replica to reach the user's last-write position.

Q3. Users edit their profile and the old version shows on refresh. Fix it. SCENARIO

The read hit a lagging replica. Add read-your-writes: route that user's reads to the primary for a few seconds or use a version token, and monitor lag.

Q4. Which features of a social app can be eventually consistent, and which can't? SCENARIO

Likes, feeds, follower counts and view counts can; payments, account balances, unique usernames and permission changes need strong consistency.

Q5. How long is 'eventually'? TRICKY

No bound is promised; in practice it's replication lag, usually milliseconds to seconds, but it can grow under load or partitions, so measure and alert on it.

Q6. Can an eventually consistent system lose data? TRICKY

Yes, with last-write-wins on concurrent writes or if a node holding unreplicated writes dies. Use quorums, durable logs and merge-friendly data types to avoid it.

Next: SD-H-019 · CRDTs